

For combinatorial circuits:

- All **if**, **else**, **if else**, **case**, **while**, and **for** statements must be in a **always_comb** block
- When using an **always_comb** block, all outputs must be declared as logic in the module description:

Ex:

```
module Example (  
    input [3:0] A,  
    output logic [4:0] F );
```

- Every **if** should have an **else** (you can have as many **else if** as you'd like)
- Every **case** statement should have a **default**
- Logical operators should be used within if, else if statements (**&&** **||** **!**) instead of bit wise operators (**&** **|** **~**)

Other useful info:

- The syntax to instantiate a module within a top level module is as follows:

```
moduleName instanceName (.portName (connectionName));
```

Ex:

```
Example example_inst (.A(t1) , .F (t2) )
```

- The signal you are connecting a port to must be declared with the same bit width as the port.
For the example above:

```
logic [3:0] t1;  
logic [4:0] t2;
```


Note the size of the signal always comes right before the signal name.

- The syntax to parameterize the bit width of inputs or outputs in a module is:

```
module Example # (parameter WIDTH = 2) (  
    input [WIDTH-1:0] A,  
    output logic [WIDTH:0] F );
```

- To instantiate a module with a parameter use the following syntax:

```
Example #(4) example_inst (.A(t1) , .F (t2) )
```

If you do not specify the value of the parameter, it will use the default value (in the example above the default value is 2)